JOINT DELIVERY ARCHITECTURE

Koard + Decionis
Agentic card authorization

Cardholder-present verification before card use, bound to a purchase intent and enforced at the bank's authorization boundary.

The joint delivery

Koard is the design partner for cardholder-present verification. Decionis provides independent execution authority. AgentSafe enforces the grant boundary in the bank; the issuer retains the final card decision.

PARTICIPANT	OWNS	OUTPUT
Agent platform	Propose the precise purchase; wait before presenting the card.	Purchase intent
Koard	Verify cardholder presence and bind the intended card to the purchase.	Signed presence evidence
Decionis	Verify evidence and apply the bank's versioned policy.	Decision and bound grant
AgentSafe	Capture, hold, match, claim once and record effect evidence.	APPROVE / PENDING / NO_MATCH
Issuer / processor	Operate the hook and account/fraud controls; authorize or decline.	Card decision and result

Three phases, with one clear point of execution

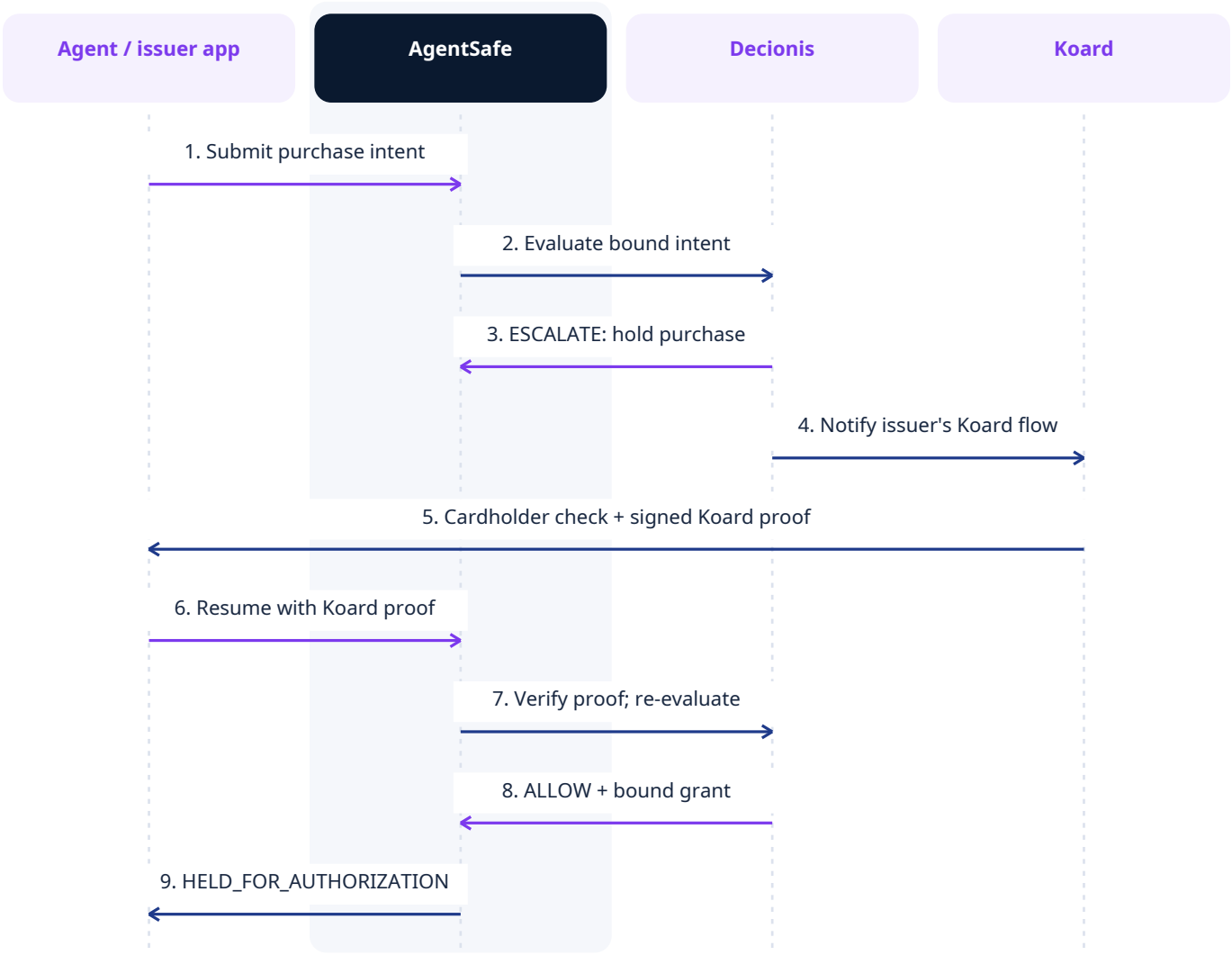
Before card use: establish intent and required presence, then obtain authority.
At authorization: match and claim within the issuer's measured deadline.
Afterward: record what the issuer actually did and reconcile uncertainty.

A tap is awaited before card submission or, when the platform can pend, in the bank's fraud queue. The match itself never waits for a tap.

PHASE 1 | BEFORE CARD USE

Verify presence, then decide

The original purchase stays immutable. Koard supplies evidence for that purchase; only a fresh policy decision can issue an executable grant.



Koard signs the verification result and returns it through the agreed issuer-app integration. The issuer app's backend presents it as an operator holding escalations.resume, or the agent that proposed presents it; either way the authority re-evaluates the same intent. A bank-role approval uses Decionis Presence instead.

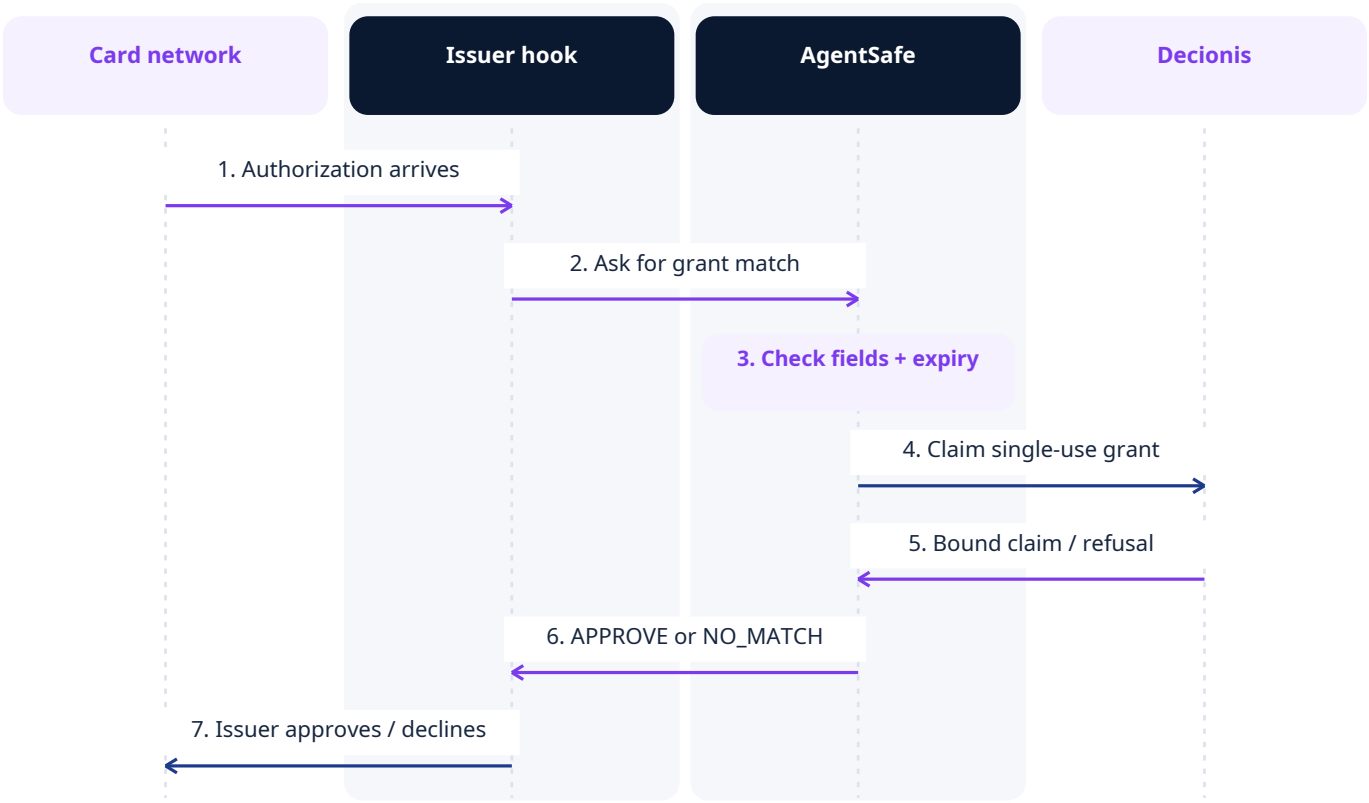
No permission is created by a tap alone

BLOCK stops submission. ESCALATE stays held. If the intent or evidence expires, obtain a fresh intent and approval as required. Successful presence must satisfy the bank's policy before ALLOW.

PHASE 2 | ISSUER AUTHORIZATION

Match locally. Claim once.

The issuer asks about the authorization it received. The matcher runs in the bank; claiming the grant is a call to the configured authority, or a local claim at the edge when the budget demands it.



Every match must satisfy the same purchase

Exact issuer token reference, currency and merchant id; exact merchant category when the purchase specified one; positive amount no greater than the granted ceiling; unexpired, unused grant. The grant is cryptographically verified and claimed against its original intent.

APPROVE is the intent leg of the issuer's decision

The issuer still applies funds, fraud and other card controls. NO_MATCH means usable authority was not established; it can reflect missing state, mismatch, expiry or claim failure. It does not prove the cardholder was absent.

Budget for the hook's answer at the 99th percentile: 150 ms with the hosted authority in the same cloud region, 300 ms across regions, 10 ms with a local claim at the edge; the in-process match is under a millisecond, and each is confirmed on the bank's own link before enforcement. Define an explicit NO_MATCH and timeout policy; strict enforcement must not silently bypass a required grant.

PHASE 2 | WHERE A TAP CAN WAIT

Three places, three taps

An authorization lives on a clock measured in seconds, so where a held item waits while a person answers is a design decision per bank. Koard's paper names three places and three taps; this is what each is on AgentSafe.

WHERE THE ITEM WAITS	KOARD'S NAME	WHAT AGENTSAFE DOES
Before the authorization exists	Tap to Verify, for a held purchase	The agent proposes, policy holds the grant or asks for the cardholder, the tap resumes the hold, the authorization matches it in milliseconds. The cleanest path, and the one built first.
In the bank's fraud queue	Tap to Verify, for a flagged authorization	EXECUTOR_CARD_NO_GRANT=evaluate: an authorization with no hold is evaluated at the commit boundary. ALLOW is held and claimed in the same call; ESCALATE is answered PENDING with the handoff, the platform pends the authorization, the tap arrives through POST /v1/escalations, and the re-presented authorization matches the hold the tap left. Needs a platform that can pend.
After the authorization	Tap to Confirm, called by the platform	Nothing can pend, so the authorization goes through and the cardholder confirms or rejects it afterwards in the issuer's app; a rejection is a reversal the bank's own rail carries. AgentSafe's part is evidence only: the same evaluation in SHADOW records what policy would have said, and the ISO 8583 interceptor's shadow mode records it on the wire. The easiest to build and the weakest: the money has moved.
A standing permission, no amount behind it	Tap to Authenticate	Not a purchase: the cardholder grants an agent permission to buy on their behalf, a new device, account recovery. The producer signs it as a standing signal the agent presents with each purchase; the authority verifies it against the producer's key and policy names its facts with required_signals.

The fraud-queue contract

A PENDING answer carries the authorization id, the intent id and hash, the decision and dossier ids, expires_at and the escalation handoff; Decionis tells the subscriber at the same moment. The issuer app's backend, an operator holding escalations.resume, presents the tap; the platform then re-presents the same authorization, which is approved, claimed once and settled by its result. Re-presented before the tap it is PENDING again from the same intent; past expires_at it is evaluated afresh; a result for a PENDING authorization is refused as unknown.

Soft decline and re-present is left off on purpose, as Koard's paper leaves it off: the step-up it invites assumes a person at a browser.

CONTRACTS | INTEGRATION HANDOFFS

Name the receiving boundary

These are separate interfaces. The issuer's hook calls AgentSafe's trusted executor; it does not call the generic proxy's control routes.

CALLER / RECEIVER	CONTRACT	PURPOSE
Agent to AgentSafe	POST /v1/actions **PROPOSER**	card.purchase with target card:<token ref>
AgentSafe to Decionis	POST /v1/authority/enforce-and-bind	Evaluate the captured intent; return a decision and grant if allowed.
Decionis to subscriber	authority.escalation_required	Notify the bank-configured endpoint for the Koard journey.
Issuer app to AgentSafe	POST /v1/escalations **OPERATOR: escalations.resume**	Resume the same intent with the signed presence attestation; the proposer may also present it.
Issuer hook to AgentSafe	POST /v1/card-authorizations **OPERATOR: cards.authorize**	Match request fields and claim one bound grant; with no hold, evaluate at the commit boundary and answer APPROVE, PENDING or NO_MATCH.
AgentSafe to Decionis	POST /v1/execution/claim-token	Consume authority once, subject to validity and current checks.
Issuer hook to AgentSafe	POST /v1/card-authorizations/{authorization_id}/result	Report the issuer result for comparison and finalization.
Bank operator to AgentSafe	POST /v1/control/halt **OPERATOR: halt**	The stop: no new work; the hook answers NO_MATCH with EXECUTOR_HALTED.

Bind the same identifiers through every handoff

Carry tenant, intent id/hash, decision, dossier and grant bindings. Use a stable issuer authorization id: identical concurrent retries share one claim and answer, and a changed request under the same id is refused. The actor submitting a purchase must not hold issuer-authorization authority: separate principals, the processor credential restricted to cards.authorize, the issuer app to escalations.resume.

EVIDENCE | PRESENCE AND EFFECT

Keep proof and outcome distinct

Koard proves presence for the purchase. The issuer reports the card decision. Decionis records the authority decision; AgentSafe links the observed effect.

Presence attestation: the Decionis-side contract

Compact JWS with alg=EdDSA, typ=decionis-presence-attestation+jwt and the registered kid. The bank admits Koard's key and issuer identity. Align Koard's native output with this contract in the joint integration.

CLAIMS	BINDING
iss, aud, sub	Registered provider; bank organization; opaque cardholder/token reference, never a PAN.
intent_id, intent_hash	The exact purchase being approved. Another purchase requires its own bound evidence.
iat, exp, jti	Issued time, expiry and unique attestation id; declared lifetime at most 300 seconds. Agree clock-skew tolerance.
presence	method=card_tap; result=VERIFIED; approver_role=cardholder. Optional card_token_ref, token_match and network_validated.

Issuer result: the current AgentSafe endpoint

```
POST /v1/card-authorizations/auth_ref_demo/result
{
  "status": "APPROVED",
  "approved_amount_minor": 4000,
  "auth_code": "A7"
}
```

The endpoint authenticates an issuer operator and accepts structured JSON. It does not accept an issuer-signed result JWS. AgentSafe builds effect evidence from this report and submits finalization; a cryptographic issuer receipt requires a separately agreed connector contract.

Approved amounts within the purchase ceiling match; excess is an effect mismatch and halts by default. DECLINED records failure. A result after the claim lease is indeterminate, with reconciliation required.

JOINT DELIVERY | ACCEPTANCE

Close the issuer integration

The core card flow exists in AgentSafe source. Production acceptance depends on the Koard ceremony, bank policy and the processor's actual authorization hook working together.

WORKSTREAM	ACCEPTANCE EVIDENCE
Koard + issuer app	Demonstrate the purchase shown to the cardholder, proof-to-card/token binding, signed handoff, failed/abandoned verification, expiry and key rotation.
Decionis + bank risk	Register the producer key or admit Koard from the directory; admit the required presence facts; prove re-evaluation after evidence; test mismatched tenant/intent and revoked keys.
Issuer / processor	Agree token and merchant mapping, real-time hook, deadline, NO_MATCH and PENDING handling, duplicate authorization handling and final result reporting.
Bank platform	Separate identities, restrict bypass paths, require durable evidence, share the hold store across replicas, exercise restart, recovery and the stop.
Joint validation	Run normal purchase, BLOCK, ESCALATE, PENDING in the fraud queue, changed amount/merchant, expired grant, duplicate claim, identifier collision and unknown outcome end to end.

Limits that shape the rollout

One pending purchase per card per merchant. Split authorizations are not offered; incremental ones are, per deployment, from the same claim within the ceiling. Rotating the hold key drains holds rather than re-sealing them. Velocity is counted by AgentSafe per card.

Where the authority and Presence run per hosting tier, and whose custody each key is in, is written once: docs/gateway/deployment.md in the AgentSafe repository and decionis.com/docs/card-authorization. A local claim at the edge removes the hook's network call when the replay store is the shared Postgres one, the evidence chain is retained and signed, and the bundle and usage-report keys live in the bank's key vault.

Review basis: Koard's workflow review of 8 October, the design-partner context and AgentSafe at its 13 October revision (holds by card and merchant, the shared hold store, the commit-boundary evaluation and the hosting answers). Decionis-side attestation, admission and managed-escalation contracts were checked in source; live service availability and issuer integration were not tested.